

Project Executable Files

Date	03 July 2026
Team ID	N/A
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	3 Marks

Project Executable Files

This section documents all executable and deployable files for OptiCrop, organized so an evaluator can run the solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt (dependency file)	Yes
4	Database schema / seed files	NA — no traditional database; uses CSV dataset + Pickle model files
5	Environment configuration file (.env.example or similar)	NA — no API keys or secrets required
6	Deployed application URL (if hosted)	NA — demonstrated via local Flask dev server (can be added if deployed to Render/Heroku)
7	APK / Executable binary	NA — web application, not a mobile/desktop app
8	Dockerfile / Containerization config	NA — not containerized for this version
9	Test files and test results	Partial — train_model.py includes a train/test split and prints evaluation metrics for all 4 models
10	Demo video or walkthrough	[To be added — link to your recorded demo]

Step 2: File / Folder Structure

```

OptiCrop/
|-- data/
|   |-- crop_data.csv          - Training dataset (2,200 rows, 22 crops)
|   |-- generate_dataset.py    - Script that generated the synthetic dataset
|-- model/
|   |-- model.pkl              - Trained best-performing model (Random Forest)
|   |-- scaler.pkl             - StandardScaler used at prediction time
|   |-- model_name.txt         - Name of the selected best model
|-- notebooks/
|   |-- OptiCrop_Model_Building.ipynb - Full EDA + preprocessing + model-building notebook
|-- templates/
|   |-- index.html             - Home page
|   |-- about.html             - About page
|   |-- find_your_crop.html    - Prediction form + result page
|-- static/
|   |-- css/style.css          - Shared stylesheet
|-- app.py                     - Flask application (routes + prediction logic)
|-- train_model.py             - Standalone training/evaluation script
|-- requirements.txt            - Python dependencies

```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	N/A — run locally via the Flask development server (can be deployed to Render/Heroku as a future step)
Login Credentials (Demo)	N/A — OptiCrop has no authentication; the app is open to any user
Platform / Hosting Provider	Local machine (Flask dev server); no cloud hosting used for this version
Repository Link	https://github.com/v-arun-kumar-03/OptiCrop
Demo Video Link	[Add your recorded demo link here]

Step 4: Run Instructions

Pre-requisites: Python 3.10+, pip

Step 1: Clone the repository: `git clone https://github.com/v-arun-kumar-03/OptiCrop.git`

Step 2: Create and activate a virtual environment: `python -m venv venv & venv\Scripts\activate`

Step 3: Install dependencies: `pip install -r requirements.txt`

Step 4: Train the model (generates model.pkl/scaler.pkl): `python train_model.py`

Step 5: Start the application: `python app.py`

Step 6: Open the browser at `http://127.0.0.1:5000`

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Flask's built-in dev server is single-threaded	Acceptable for local demo; recommend Gunicorn/WSGI + Nginx for production
2	No authentication or rate-limiting on the prediction endpoint	Acceptable for a public, stateless demo tool; can be added later if needed
3	Training data is a representative synthetic dataset, not large-scale real-world sensor data	Structured to match a real Kaggle crop-recommendation dataset; can be swapped for real data without code changes